Классификация текста

От слов в сообщении к решению модели

Сначала поставим узкую задачу

Слайд 1. Классификация текста

На прошлом занятии мы подготовили обращения в электронную приёмную. Теперь попробуем автоматически выбирать для них отдел. Для сообщения «Как продлить книгу?» ожидается библиотека, для «Не могу войти в личный кабинет» — техническая поддержка. В нашем наборе шесть категорий, и каждой строке присвоена одна метка.

Такая задача называется классификацией. У нас есть объекты — тексты обращений, известные ответы для части объектов и правило, которое должно научиться предсказывать ответ для нового сообщения. Мы не будем сразу генерировать письмо студенту: сначала получим более простой результат, категорию.

Можно начать с ручных правил. Если встречается «книга», выбирать библиотеку; если «пароль» — ИТ. Это быстро даёт понятную программу. Но затем появляется «монография», «читательский кабинет» или вопрос, в котором упомянуты два отдела. Список правил приходится расширять, а их конфликты — разрешать вручную.

В обучаемой модели часть этой работы выполняется по примерам. Мы задаём способ представить текст числами, выбираем семейство моделей и показываем сообщения вместе с метками. Алгоритм подбирает параметры так, чтобы лучше согласовать предсказания с обучающими ответами. После этого параметры используются для новых текстов.

Слайд 2. При обучении известны ответы

Важно различать обучение и применение. Во время обучения программа видит и сообщение, и его категорию. При предсказании ей передают только сообщение. Если для обработки проверочной строки понадобилась её правильная метка, значит, мы уже решили другую задачу.

Сегодня построим систему, в которой можно проследить каждый шаг: TF-IDF и логистическую регрессию. Преимущество такого начала — можно посмотреть на словарь, веса признаков и конкретные ошибки. Эти же вопросы останутся полезны, когда позже мы будем получать признаки с помощью предобученного энкодера.

Что оставить для проверки

Слайд 3. Проверяем на новых ситуациях

Если обучить модель на всех примерах и проверить на них же, результат будет рассказывать прежде всего о знакомых ей данных. Нас интересуют новые обращения. Поэтому часть корпуса нужно отложить до обучения.

Обычно выделяют train, validation и test. На train подбираются параметры. Validation помогает выбирать настройки: например, использовать признаки по словам или по символам. Test нужен для оценки уже выбранного решения. Эти части различаются назначением, а не только названием файла.

Представим, что мы сравнили десять настроек по test, выбрали лучшую и сообщили её тестовый результат как итог. Проверочные ответы уже повлияли на выбор. Даже если ни одна тестовая строка не участвовала непосредственно в `fit`, набор стал частью разработки. Чтобы проверить, как выбранная модель работает на новых данных, теперь понадобится новая отложенная выборка.

В нашем корпусе есть ещё одна особенность: пять формулировок могут относиться к одному базовому обращению. Если распределить строки случайно, похожие варианты одной ситуации могут попасть по обе стороны разделения. Модель увидит на проверке почти знакомый вопрос.

Поэтому распределяем по частям целые группы с одинаковым `group_id`. В наборе 120 групп: 72 в train, 24 в validation и 24 в test. Вместе с пятью вариантами каждой группы это 360, 120 и 120 строк. Проверка пересечения групп должна дать пустое множество для каждой пары частей.

Одновременно стоит посмотреть на категории. Если какой-то класс отсутствует в проверке, полученная метрика ничего не скажет о его распознавании на этом наборе. Если один отдел представлен гораздо чаще других, простой общий процент правильных ответов может скрыть ошибки на редких категориях. К этому мы вернёмся после обучения первого варианта.

Превратим слова в координаты

Слайд 4. Каждому слову — своя координата

Классификатору понадобится числовое представление сообщения. Самый простой вариант — выбрать словарь и посчитать, сколько раз каждое слово встретилось в тексте. Одно слово соответствует одной координате. Такой способ называют мешком слов: порядок в пределах сообщения не сохраняется.

Возьмём три коротких документа:

```
книга книга библиотека
книга продление
общежитие оплата
```

Словарь состоит из пяти слов. В порядке «библиотека», «книга», «общежитие», «оплата», «продление» первый документ получает вектор (1;2;0;0;0), второй — (0;1;0;0;1), третий — (0;0;1;1;0).

В этих векторах уже видна связь первых двух документов: у них ненулевая координата «книга». Третий использует другие слова. Мы заменили строки одинаковыми по размеру числовыми объектами, которые можно сравнивать и подавать модели.

Размер словаря может быть большим, хотя в каждом сообщении встречается лишь небольшая его часть. Поэтому большинство координат равны нулю. Такие матрицы удобно хранить в разреженном виде — записывать ненулевые значения и их положения. Преобразование большой матрицы в обычный плотный массив без необходимости расходует память на нули.

Можно считать не только отдельные слова, но и соседние пары — биграммы. Тогда сочетание «не работает» станет самостоятельным признаком. Это частично возвращает информацию о локальном порядке, потерянную обычным мешком слов. Более длинные фрагменты точнее описывают конкретные формулировки, но встречаются реже и увеличивают словарь.

Попробуйте представить два сообщения «работает портал» и «не работает портал» сначала униграммами, затем униграммами вместе с биграммами. Какие координаты совпадут, а какие помогут их различить? Здесь видно, почему сохранение отрицания на предыдущем этапе было важно.

TF-IDF меняет вклад частых слов

Слайд 5. TF и DF считают разные вещи

Обычный счётчик одинаково учитывает два слова с одной и той же частотой внутри документа. Но одно из них может встречаться почти в каждом обращении, а другое — только в текстах определённой темы. TF-IDF объединяет частоту слова в текущем документе и его распространённость по коллекции.

TF обозначает частоту терма. В нашем ручном примере это просто число появлений слова в документе. DF считает другое: в скольких документах слово встретилось хотя бы один раз. Для «книги» DF равно 2: слово есть в первом и втором документах. Два появления в первом дают TF=2, но при подсчёте DF этот документ учитывается один раз.

Слайд 6. Меньше DF — больше IDF

IDF становится меньше для слов, встречающихся во многих документах. Возьмём сглаженную формулу scikit-learn. Здесь N — число документов, а df(t) — число документов со словом t:

```
idf(t) = ln((1 + N) / (1 + df(t))) + 1
tfidf(t, d) = tf(t, d) · idf(t)
```

У нас N=3. Для «книги» получаем `1 + ln(4/3) ≈ 1,2877`. У каждого из остальных четырёх слов df=1, поэтому IDF равна `1 + ln(4/2) ≈ 1,6931`. В первом документе ненормированные веса «библиотеки» и «книги» составляют примерно 1,6931 и 2,5754. Повтор слова повлиял на его вес.

Слайд 7. Длина вектора становится единицей

Затем выполним L2-нормировку: разделим все координаты вектора на его евклидову длину. Первый документ после этого имеет ненулевые координаты примерно 0,5494 и 0,8356. Его длина равна единице, но сумма координат не обязана быть единицей. Это веса признаков, а не распределение вероятностей по словам.

Нормировка помогает отделить направление вектора от общего масштаба частот. Два текста с похожим относительным составом слов могут оказаться близкими, даже если один существенно длиннее. Насколько это полезно для нашей задачи, выясняется при сравнении моделей.

Теперь вопрос об обучающей части. Словарь, N и df должны определяться по train. При обработке validation и test применяется уже готовое преобразование. Если заново посчитать IDF по всему корпусу, проверочные тексты повлияют на признаки, с которыми работает модель.

Неизвестное слово не получит новой координаты во время `transform`. Если ни один фрагмент нового сообщения не попал в словарь, вектор может оказаться нулевым. Это конкретный случай, который стоит проверить: какая информация тогда останется у классификатора и что определит его ответ?

Как линейная модель выбирает класс

Слайд 8. Побеждает наибольшая оценка

После векторизации каждый текст — вектор x. Линейный классификатор назначает каждому классу набор коэффициентов w и смещение b. Оценка класса вычисляется как скалярное произведение плюс смещение:

```
s_c = w_c^T x + b_c
```

Для нового текста выбирается класс с наибольшей оценкой. В логистической регрессии эти оценки связаны с вероятностной моделью, а параметры подбираются по размеченным примерам. Слово «регрессия» в названии здесь не означает, что мы предсказываем число вместо категории: это стандартный алгоритм классификации.

Разберём маленький пример с двумя признаками, «книга» и «пароль». Пусть их значения в сообщении равны (0,8;0,1). Для библиотеки зададим коэффициенты (2;−1), для ИТ — (−1;2), смещения примем равными нулю. Оценка библиотеки будет `2·0,8 − 0,1 = 1,5`, оценка ИТ — `−0,8 + 2·0,1 = −0,6`. Выбирается библиотека.

Эти коэффициенты мы придумали для расчёта. Как алгоритм поймёт, что нашёл более удачные? Для обучения нужно связать предсказание с известной меткой и измерить ошибку. Вернёмся к сообщению о книге: правильная категория в обучающей таблице — библиотека.

Слайд 9. Softmax даёт вероятности

В нашем опыте с шестью категориями логистическая регрессия превращает оценки классов в вероятности с помощью softmax: для каждого класса берётся exp(s_c), затем результат делится на сумму экспонент всех оценок. Для ручного расчёта оставим два класса. При наших оценках 1,5 и −0,6 библиотека получает вероятность около 0,891, ИТ — около 0,109. Большая оценка стала большей вероятностью, а две вероятности складываются в 1.

Слайд 10. Уверенная ошибка стоит дороже

Обозначим вероятность правильной категории через p_y. Потеря одного примера равна −ln(p_y). Если p_y=0,9, потеря около 0,105; если p_y=0,1 — около 2,303. Таким образом, уверенный ответ в пользу неверного класса получает большой штраф. Правильная метка нужна здесь, чтобы выбрать, какую вероятность оценивать.

Обучающий алгоритм вычисляет, как изменение параметров влияет на потерю, и делает шаг в сторону её уменьшения. Такие производные по параметрам называют градиентом. Шаги повторяются по обучающим примерам, пока не выполнится условие остановки. Получается последовательность: текст → признаки → оценки → вероятности → потеря по известной метке → изменение коэффициентов.

Слайд 11. Новый вес — новая оценка

Посмотрим на один возможный шаг. Если прибавить 0,1 к коэффициенту признака «книга» у библиотеки, его вклад при x=0,8 вырастет на 0,08. Оценка библиотеки станет 1,58 вместо 1,5. При неизменной оценке ИТ вероятность правильного класса повысится примерно с 0,891 до 0,898, а потеря данного примера уменьшится. В настоящем обучении изменение согласуется с потерями остальных сообщений и регуляризацией, поэтому нельзя независимо увеличивать каждый понравившийся коэффициент.

Слайд 12. Регуляризация ограничивает веса

Параметры должны одновременно объяснять разные сообщения. Если очень точно подстроиться под случайные особенности train, перенос на новые обращения может ухудшиться. Поэтому помимо ошибки на train мы следим за качеством на отложенной части и ограничиваем сложность решения.

Регуляризация помогает ограничить такую подстройку. Она заставляет учитывать не только ошибку на обучающих примерах, но и величину коэффициентов. В используемой реализации параметр C связан с обратной силой регуляризации: меньший C означает более сильное ограничение. В основной лабораторной мы сохраняем один C для двух вариантов признаков, чтобы сравнивать именно представления текста.

Коэффициенты можно изучать, но их смысл относится к данной модели и её признакам. Положительный вес слова для класса показывает вклад в линейную оценку при прочих заданных координатах. Он не доказывает, что это слово само по себе определяет истинную категорию любого сообщения.

С чем сравнить первый результат

Слайд 13. Baseline задаёт точку отсчёта

Перед сложной системой полезно проверить совсем простую: всегда возвращать одну категорию. Такой baseline задаёт точку отсчёта. Он не читает сообщение, зато показывает, сколько правильных ответов можно получить только за счёт состава выборки.

Если шесть категорий встречаются одинаково часто, постоянный ответ угадает примерно одну шестую объектов, то есть 16,7%. Если 90% писем относятся к ИТ, правило «всегда ИТ» получит 90% правильных ответов. Во втором случае число выглядит внушительно, хотя модель по-прежнему ничего не извлекает из текста.

Поэтому результат обученной системы нужно читать рядом с baseline и распределением классов. Само число 90% не позволяет понять, научилась ли программа решать задачу. Важно, какие сообщения она распознаёт, какие пропускает и насколько это отличается от простой стратегии.

В коде такую модель можно получить через `DummyClassifier(strategy="most_frequent")`. Его обучают на train, чтобы определить наиболее частую категорию, и затем применяют к validation. Если частоты равны, конкретная выбранная метка зависит от принятого порядка; общая доля верных ответов при равных категориях от этого не меняется.

Для нашего первого опыта baseline также проверяет весь путь оценки. Если простая стратегия неожиданно получает другой результат, стоит посмотреть на метки,

размеры частей и функцию подсчёта. Иногда проблема находится в подготовке сравнения, а не в обучаемой модели.

Какие ошибки скрывает общий процент

Слайд 14. Accuracy может скрыть провал

Accuracy — доля объектов с правильной меткой. Это понятная величина: из ста сообщений верно направлены восемьдесят — accuracy равна 0,8. Но она не различает, в каких категориях произошли двадцать ошибок.

Матрица ошибок показывает это подробнее. Зафиксируем соглашение: строки — настоящие классы, столбцы — предсказанные. Диагональ содержит правильные ответы. Если на пересечении строки «библиотека» и столбца «ИТ» стоит большое число, библиотечные обращения часто отправляются в поддержку.

Посчитаем пример с 90 объектами A и 10 объектами B. Модель всегда отвечает A. В матрице строка A содержит 90 в столбце A и 0 в столбце B; строка B — соответственно 10 и 0. Accuracy равна 0,9, хотя ни один объект B не распознан.

Слайд 15. Precision и recall считают разное

Для одного выбранного класса можно выделить true positives, false positives и false negatives. TP — правильно найденные объекты этого класса. FP — объекты других классов, ошибочно отнесённые к нему. FN — объекты класса, которые модель отправила в другие категории.

Precision отвечает на вопрос: какая доля предсказаний выбранного класса оказалась правильной? Recall — какая доля всех настоящих объектов этого класса была найдена? Формулы различаются знаменателем:

```
precision = TP / (TP + FP)
recall = TP / (TP + FN)
```

Для класса A в нашей матрице TP=90, FP=10, FN=0. Из 100 предсказаний A верны 90: precision=0,9. Все 90 настоящих объектов A найдены: recall=1.

Слайд 16. Macro-F1 даёт классам равный вес

F1 объединяет precision и recall:

```
F1 = 2TP / (2TP + FP + FN)
```

Вернёмся к нашей матрице. Для A получаем F1=180/190≈0,9474. Для B ничего не найдено, F1=0. Среднее по двум классам — примерно 0,4737. Высокая accuracy соседствует с полным провалом распознавания B.

Macro-F1 учитывает показатели всех классов. Сначала F1 рассчитывается отдельно по каждому классу, затем среднее этих значений с одинаковым весом классов даёт macro-F1. Поэтому редкая категория влияет на него заметнее, чем на общую accuracy. Эти две метрики могут совпасть, но в общем случае измеряют разные стороны результата.

При отсутствии предсказаний класса precision имеет нулевой знаменатель. В лабораторной зададим явное соглашение `zero_division=0`. Полный список шести меток зафиксирует состав усреднения, в том числе если какой-либо класс отсутствует и среди истинных меток, и среди предсказаний. Если класс есть в правильных ответах, отсутствие его предсказаний само по себе не исключает этот класс из macro-F1.

Выбор метрики зависит от применения. Если важно обслуживать все отделы, macro-F1 и показатели по классам дают полезную информацию. Если ошибки имеют разную цену, потребуется дополнительно учитывать эту цену. Одной метрики не всегда достаточно, но для конкретного сравнения нужно заранее выбрать основной критерий.

Слова или символьные фрагменты

Слайд 17. Словоформы имеют общие части

Теперь сравним два представления. Первое использует отдельные слова и соседние пары. Второе — символьные фрагменты длиной от трёх до пяти символов внутри границ слов, режим `char_wb`.

Слова удобны для чтения человеком: можно открыть список признаков и увидеть знакомые выражения. Но «книга» и «книгу» оказываются разными словными элементами. У символьных признаков часть фрагментов совпадёт, например «кни» и «ниг». Это может помочь работать со словоформами и некоторыми опечатками.

Символьный подход также получает множество коротких фрагментов, которые по отдельности трудно интерпретировать. Он не знает, что «сайт» и «портал» могут обозначать похожие вещи: между этими словами почти нет общих буквенных частей. Полезные связи должны проявиться через размеченные примеры и общую структуру данных.

Слайд 18. Выбираем по validation

Чтобы понять вклад представления, оставим классификатор одинаковым. Обучим вариант по словам и вариант по символам на одном train, оценим на одном validation по одной функции метрик. Если одновременно сменить представление, алгоритм и разбиение, будет трудно объяснить, что именно улучшило результат.

В проверенном учебном опыте на validation получились следующие значения:

Представление	Accuracy	Macro-F1
Слова и биграммы	0,4500	0,4120
Символьные фрагменты 3–5	0,7417	0,7430

В основной работе выбор производится по macro-F1 на validation. По этому критерию выбран символьный вариант. Это результат нашего корпуса и настроек, а не обещание превосходства символьных признаков на любой задаче. После выбора применяем выбранный вариант к test. Если захотим дальше разрабатывать признаки, это уже следующий опыт: записываем изменение и сохраняем разделение между разработкой и окончательной проверкой.

Различайте два критерия: внутри `fit` логистическая регрессия оптимизирует логарифмическую потерю с регуляризацией, а macro-F1 на validation используется нами для выбора между представлениями. Это связанные, но разные этапы работы.

Зачем объединять обработку в Pipeline

Слайд 19. Pipeline сохраняет порядок действий

У нашей системы два связанных шага: преобразователь текста и классификатор. При обучении сначала строится словарь и вычисляется IDF, затем полученная матрица вместе с метками подаётся классификатору. При предсказании новый текст проходит через тот же преобразователь с уже сохранёнными параметрами.

`Pipeline` объединяет эти действия в один объект. Вызов `fit` обучает последовательность на переданных данных, а `predict` выполняет преобразование и предсказание. Так удобнее сохранить правильный порядок и избежать случайного обучения векторизатора на другой части корпуса.

В нашем опыте validation и test проходят через уже обученную систему: для этих частей мы вызываем `predict`, а не `fit`. Преобразователь не должен создавать для них новый словарь: тогда координаты изменятся и обученные коэффициенты перестанут соответствовать тем же признакам.

Слайд 20. Сначала split, затем fit

Рассмотрим ошибку: сначала вызвать `fit_transform` на всех текстах, а потом разделить готовую матрицу. Метки теста могли не использоваться, но его словарь и частоты уже повлияли на признаки. Для нашей оценки классификатора такая подготовка раскрывает информацию из отложенной части.

В поиске по заранее известной коллекции постановка будет другой: документы составляют саму доступную базу. На следующей лекции мы сможем построить индекс по всей этой известной базе. Важно различать эти задачи, а не запоминать

правило «никогда не обрабатывать все документы» без связи с назначением проверки.

Число метрики — начало разбора

Слайд 21. На тесте верны 95 ответов из 120

После итогового предсказания откроем ошибки. В таблице нужны исходное сообщение, правильная категория, ответ модели и `group_id`. Несколько стилистических вариантов одной ситуации стоит рассматривать вместе: у пяти похожих ошибочных ответов может быть одна и та же причина.

В проверенном символьном варианте учебного опыта accuracy на test получилась около 0,792, macro-F1 — около 0,765. Правильные ответы получены для 95 из 120 строк. Эти 120 строк относятся к 24 исходным ситуациям: общий процент не заменяет разбор групп.

Слайд 22. Число метрики не объясняет ошибку

Среди ошибок есть обращение «Не могу найти монографию в списке доступной литературы.». Его правильная категория — библиотека, но модель выбрала ИТ.

Можно предположить, что модель недостаточно связала «монографию» с библиотечной темой. Чтобы проверить объяснение, стоит посмотреть на обучающие примеры и признаки, а затем провести отдельное сравнение. Одного знакомого слова в ошибочном сообщении недостаточно, чтобы уверенно назвать причину решения.

Слайд 23. Одна просьба — разные ответы

При этом ответы внутри одной группы не всегда одинаковы. Фраза «Мне нужна помощь в подборе источников для курсовой.» была направлена в ИТ. Та же просьба с приветствием «Здравствуйте!» попала в учебный процесс. Все пять вариантов этой группы оказались ошибочными, но предсказанные категории различались: один ответ ИТ и четыре — учебный процесс. Это повод проверить устойчивость к формулировке, а не только посчитать ошибки группы. В этом сравнении модель получает текст вместе с точкой в конце: если убрать её из исходной просьбы, тот же классификатор уже выбирает учебный процесс. Поэтому для повторения результата сохраняем строку целиком, вместе с пунктуацией.

Другой пример — «Как исправить ошибку в зачётной ведомости?». Здесь слово «ошибка» встречается в учебном, а не техническом контексте. Такой случай полезен для анализа границ классов и того, какое окружение учитывает представление. На следующих занятиях мы сравним его с контекстными признаками энкодера.

В лабораторной вам предстоит получить собственные таблицы метрик и прочитать несколько разных ошибочных групп. В выводе укажите выбранное представление, результаты сравнения и то, что заметили в конкретных обращениях. Предложение следующего изменения должно опираться на эти наблюдения: например, проверить другие признаки или улучшить разметку спорных случаев.

Теперь у нас есть первая обученная система и способ понять её результат. На следующей лекции векторы будут служить ещё одной задаче — поиску подходящих документов. Геометрия представления станет видна непосредственно в порядке выдачи.

Для дальнейшего чтения

– scikit-learn: извлечение признаков из текста.

– scikit-learn: Pipeline.

– scikit-learn: метрики классификации.

– Jurafsky, Martin. Speech and Language Processing.